Reporte

Nombre: Osmel De Jesús Pérez Carrizales José Osvaldo Barcenas González Gerzayn Flores Rodriguez	**Matrícula**: 2157401 2158657 2160879
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 24	**Actividad**: Actividad Unidad 3
Fecha: 12/10/2026	
Bibliografia (Formato APA)	

Objetivo:

Convertir el análisis de negocio de **PetMarket** de la Unidad 2 (BRD, 21 historias de usuario, glosario y BPMN) en una propuesta de arquitectura de software coherente que separe estrictamente el dominio, la aplicación y la infraestructura, determinando con precisión dónde vive cada regla de negocio (invariantes, reservas de 15 minutos, revalidación de stock, división de pedidos en sub-envíos, revisión antifraude, cancelaciones y devoluciones) antes de programar el sistema.

Procedimiento: Diseño del Sistema

Caso: PetMarket

Esta actividad continúa el análisis de negocio que realizamos en la Unidad 2 para PetMarket, una plataforma de comercio electrónico de comestibles, complementos y artículos para mascotas que opera con dos depósitos y múltiples modalidades de entrega. En la Unidad 2 definimos las necesidades mediante un BRD, 21 historias de usuario con criterios Gherkin, un glosario de lenguaje ubicuo y el diagrama BPMN del flujo completo (desde la intención de compra hasta la entrega, cancelaciones y devoluciones). En esta unidad tomamos la decisión arquitectural fundamental: **decidir dónde vive cada regla de negocio y cómo colaboran los componentes del sistema para proteger las invariantes**.

Trabajamos en las nueve etapas que marca la guía de la actividad:

1. Preparar el caso y seleccionar casos de uso.

2. Elegir el patrón de lógica de negocio (*Domain Model*).
3. Construir el mapa de dominio (Aggregates, Value Objects, Domain Services y Events).
4. Diseñar la Service Layer (Application Services).
5. Separar persistencia y transacciones (Repository y Unit of Work).
6. Separar comandos y consultas (CQRS básico).
7. Revisar la propuesta con los principios SOLID.
8. Representar la arquitectura mediante diagramas UML (Clases y Secuencias) y registros ADR.
9. Validar y entregar la propuesta.

Utilizamos inteligencia artificial como copiloto para proponer, comparar y verificar alternativas, contrastando cada respuesta contra el BRD y la guía de trabajo. Los hallazgos y decisiones del equipo se encuentran registrados en la bitácora de IA.

1. Preparación del Caso

En esta etapa convertimos los productos de la Unidad 2 en la base arquitectónica. El BRD delimita el alcance: no modificamos las reglas aprobadas ni inventamos políticas nuevas. Cuando detectamos imprecisiones o cabos sueltos, los registramos como hallazgos.

- **Proceso principal:** El ciclo de vida completo de PetMarket, abarcando la exploración en carrito, la reserva y checkout, la revalidación de stock, la pasarela de pagos con revisión antifraude, la asignación de almacén y división en sub-envíos, la preparación y empaque, el envío, las cancelaciones y el procesamiento de devoluciones.

Seleccionamos **cinco casos de uso** clave que cubren flujos normales y caminos de excepción o rechazo:

- **CU-01 Registrar checkout y reservar stock:** Bloquea 15 min el stock al iniciar checkout (RN-04). Rechaza si se exceden 50 unidades por SKU (RN-07).
- **CU-02 Confirmar pago y revalidar stock:** Revalida existencias antes del cobro (RN-05). Maneja hasta 3 intentos en 15 min (RN-08) y envía a revisión

antifraude si excede $15,000 MXN (RN-09). Si se agota, presenta 3 opciones (RN-06).

- **CU-03 Asignar almacén y dividir pedido:** Asigna almacén por cercanía y carga (RN-14). Si ningún almacén cubre el 100%, divide en sub-envíos con guías independientes sin costo extra (RN-16).
- **CU-04 Cancelar pedido:** Permite autocancelación libre en app dentro de 2 horas (RN-11). Permite cancelación asistida por agente antes de "En tránsito" (RN-12). Exige autorización de supervisor si ya está "En tránsito" (RN-13).
- **CU-05 Procesar devolución:** Procesa devolución estándar dentro de 5 días hábiles con producto sin usar (RN-20). Exige evidencia fotográfica y aprobación de supervisor para devoluciones excepcionales o por daño (RN-21).

Tabla de Casos de Uso

ID	Caso de Uso	Actor	Objetivo	Información de Entrada	Reglas Relacionadas	Resultado Esperado
CU-01	Registrar checkout y reservar stock	Cliente	Bloquear existencias por 15 min en checkout	Carrito, ítems, cantidades, dirección de entrega	RN-01, RN-04, RN-07	Stock reservado por 15 min; disponible disminuye. Rechazo: >50 unidades por SKU.
CU-02	Confirmar pago y revalidar stock	Cliente / Pasarela de Pago	Validar stock final y cobrar	CheckoutId, datos de pago tokenizados, monto	RN-05, RN-06, RN-08, RN-09	Pago confirmado o revisión antifraude (> $15k). Rechazo: stock agotado (presenta 3 opciones) o

ID	Caso de Uso	Actor	Objetivo	Información de Entrada	Reglas Relacionadas	Resultado Esperado
						3er intento fallido.
CU-03	Asignar almacén y dividir pedido	Sistema / Almacén	Determinar origen del pedido	Domicilio del cliente, existencias por almacén	RN-14, RN-15, RN-16	Almacén asignado por cercanía/carga; o división automática en sub-envíos con guías propias sin cargo adicional.
CU-04	Cancelar pedido	Cliente / Agente / Supervisor	Anular pedido según estatus	PedidoId, motivo, solicitante, autorizador	RN-11, RN-12, RN-13	Pedido cancelado y stock liberado. Autocancelación (<2h) o asistida. Rechazo: "En tránsito" sin autorización de supervisor.

ID	Caso de Uso	Actor	Objetivo	Información de Entrada	Reglas Relacionadas	Resultado Esperado
CU-05	Procesar devolución	Cliente / Agente / Supervisor	Retornar producto y reembolsar	PedidoId, motivo, estado producto, evidencia fotográfica	RN-17, RN-18, RN-19, RN-20, RN-21	Devolución registrada y reembolso procesado. Rechazo: fuera de política sin evidencia ni autorización de supervisor.

Tabla de Hallazgos

ID	Hallazgo	Tratamiento	Estado
H-01	Agente de Almacén no puede reasignar almacén manualmente (RN-14).	La asignación es 100% automática en el dominio; solo el supervisor de almacén resuelve faltante físico (RN-15).	Resuelto
H-02	El Cliente en US-19 consulta estatus sin interactuar con agentes.	Se implementa modelo de lectura (Query/DTO) expuesto en app web/móvil.	Resuelto

ID	Hallazgo	Tratamiento	Estado
H-03	División de pedido genera sub-envíos independientes con folios y guías propias (RN-16).	Se modela la entidad SubEnvio dentro del agregado Pedido; el pedido padre se marca "Entregado" cuando todos sus sub-envíos llegan.	Resuelto
H-04	El monto mayor a $15,000 MXN requiere revisión antifraude de hasta 24h (RN-09).	Se define estado "En Revisión Antifraude" en el Pedido que congela el pase a preparación.	Resuelto
H-05	Las 3 opciones ante producto agotado (eliminar, esperar, sustituir) requieren confirmación explícita (RN-06).	Se implementa flujo condicional sin sustitución automática.	Resuelto
H-06	Cancelación antes de "En tránsito" permite decidir si el producto vuelve a inventario o es merma (RN-12).	El agente registra el destino del ítem al cancelar.	Resuelto
H-07	Devolución excepcional exige evidencia fotográfica obligatoria (RN-21).	Se agrega atributo de evidencia en la solicitud y validación de rol Supervisor.	Resuelto
H-08	El límite de 50 unidades aplica por	Invariante validada en el agregado	Resuelto

ID	Hallazgo	Tratamiento	Estado
	SKU por pedido (RN-07).	CheckoutSession y Pedido.	
H-09	Reintentos de pago máximos de 3 en 15 minutos (RN-08).	Se lleva un contador de intentos en la sesión de checkout/pago.	Resuelto
H-10	Si ambos intentos de entrega fallan, se ofrece reprogramación o reembolso (RN-17, RN-18).	Se gestiona el evento de entrega fallida y contador de días hábiles.	Resuelto
H-11	La U2 no especificaba el mecanismo de tokenización de tarjetas.	Se asume que la pasarela externa entrega un token y PetMarket no almacena números de tarjeta.	Supuesto validado
H-12	Los estados del pedido se derivan de la combinación del BRD y BPMN.	Se estandarizan: Pago Confirmado, En Preparación, Empacado, En Tránsito, Entregado, Cancelado.	Resuelto

2. Selección del Patrón de Lógica de Negocio

Seleccionamos **Domain Model** como patrón principal para la arquitectura de PetMarket, descartando *Table Module* (limitado a reportes tabulares) y evitando *Transaction Script* (que provocaría alta duplicación de código en las 21 reglas complejas).

Las razones que más pesaron fueron las siguientes. Primero, varias reglas dependen del estado del pedido: revalidar stock inmediatamente antes de pagar (RN-05), no cancelar sin supervisor si está "En tránsito" (RN-13), y no marcar "Entregado" el pedido padre hasta que todos sus sub-envíos lleguen (RN-16). Con Transaction Script esa condición se repetiría en cada flujo; con Table Module no habría un lugar natural para el ciclo de vida del pedido. Segundo, las mismas reglas deben cumplirse igual sin importar si el checkout se realiza por la app móvil o el sitio web. Tercero, las políticas de devolución y antifraude pueden cambiar con frecuencia. El costo que aceptamos es mayor estructura inicial que con scripts; lo acotamos con pocos agregados y sin convertir cada tabla en una entidad.

Tabla Comparativa de Patrones

Criterio del Caso	**Transaction Script**	**Table Module**	**Domain Model**	**Evidencia U2 / BRD**
Complejidad de reglas	Con 21 reglas en 5 casos, cada script repetiría validaciones de stock, tiempos y estados.	Reglas atadas a tablas de datos, no a comportamientos transaccionales complejos.	Cada regla vive en la entidad u objeto del dominio que la protege.	RN-04, RN-05, RN-09, RN-13, RN-16, RN-21.
Cambios esperados	Un cambio en la ventana de cancelación o umbral antifraude obliga a editar varios scripts.	Cambio localizado por tabla, pero mezclado con consultas SQL.	El cambio queda encapsulado en una sola clase o agregado.	BRD Sección 8 (parametrización a futuro de $15,000 MXN y 15 min).
Interacción entre conceptos	Pedido, Reserva, Almacén, Sub-envío, Pago y Devolución se cruzan en procedimientos largos.	Cada módulo ve su tabla; la interacción entre sub-envíos e inventario queda fuera.	Los objetos colaboran protegiendo sus propias invariantes.	RN-16 (Pedido padre + Sub-envíos + Almacenes distintos).

Criterio del Caso	Transaction Script	Table Module	Domain Model	Evidencia U2 / BRD
Estados e invariantes	Hay que releer el estado en cada script; riesgo de inconsistencia en inventario.	Sin modelo de ciclo de vida del pedido ni protección de reserva temporal.	Estado y transiciones completamente protegidos por los agregados.	BPMN PetMarket (revalidación stock, división, entregas fallidas).
Adicional 1: Misma regla en varios canales	Riesgo de duplicar reglas entre App y Web.	Posible, pero atado a consultas tabulares.	Ideal; la regla se ejecuta en el dominio independientemente del canal.	BRD Sección 1 (Checkout App / Web).
Adicional 2: Consultas y reportes	Posible.	Ideal para reportes diarios de discrepancia y KPIs mensuales.	Pesado para solo leer datos.	BRD Sección 10 (Reportes e indicadores).

Ventajas, Riesgos y Consecuencias

Patrón	Ventajas	Riesgos	Consecuencia en el Caso
Transaction Script	Rápido y claro para operaciones sencillas e independientes.	Duplica lógica al crecer y mezcla reglas; peligroso en reservas concurridas.	Si se usara para el pedido, RN-04, RN-05 y RN-16 se repetirían en cada flujo y generarían sobreventa de stock.

Patrón	Ventajas	Riesgos	Consecuencia en el Caso
Table Module	Bueno para reportes y manipulación de datos tabulares.	No modela el ciclo de vida del pedido ni la relación padre / sub-envíos.	Las reglas terminarían en procedimientos almacenados o servicios acoplados a la base de datos.
Domain Model	Reglas, estado e inventario juntos; una regla vive en un solo lugar seguro.	Más clases y estructura inicial; requiere mapeo ORM.	Costo inicial mayor, mitigado limitando los agregados (Pedido, Inventario, Devolucion) y usando Repository + Unit of Work.

3. Mapa de Dominio

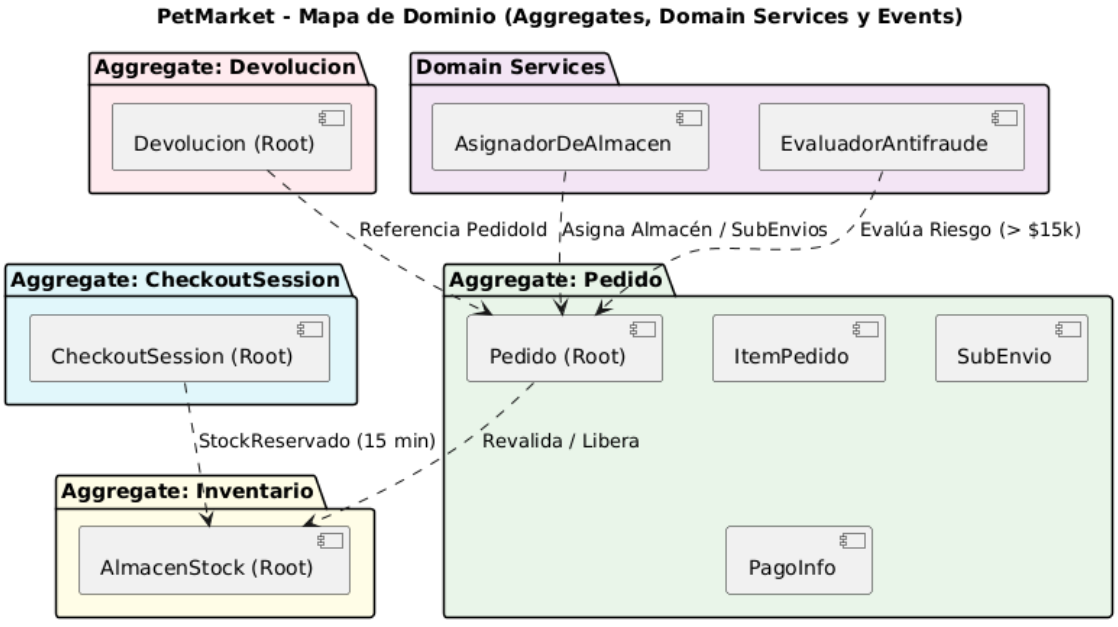
Aplicamos Domain-Driven Design: primero entender el negocio y después diseñar el software. En esta etapa identificamos los conceptos del negocio y los clasificamos como entidades, value objects, aggregates, domain services y domain events, usando el lenguaje del glosario de la Unidad 2 (lenguaje ubicuo). Un aggregate establece la frontera donde las reglas deben mantenerse consistentes, por lo que no convertimos cada tabla en una entidad.

Obtuvimos lo siguiente:

- **Cinco Aggregates:** Pedido (raíz), Inventario (raíz: AlmacenStock), Devolucion (raíz: Devolucion), CheckoutSession (raíz: CheckoutSession) y PagoRiesgo (raíz: EvaluacionRiesgo).
- **Value Objects:** SKU, Dinero, Direccion, GuiaRastreo, MetodoPagoTokenizado, CantidadStock.
- **Domain Services:** AsignadorDeAlmacen (coordina cercanía y carga de pedidos) y EvaluadorAntifraude (evalúa umbrales de riesgo).
- **Domain Events:** StockReservado, ReservaLiberada, PagoConfirmado, PedidoDividido, PedidoCancelado, DevoluciónSolicitada.

Descripción del Diagrama del Mapa de Dominio

El mapa de dominio representa visualmente las fronteras de consistencia del sistema PetMarket. Cada recuadro punteado delimita un **Aggregate** conducido por su **Aggregate Root** (Entidad Raíz). Las relaciones entre agregados nunca se realizan por referencia directa de objetos, sino exclusivamente mediante referencias por identificador único (PedidoId, AlmacenId, SKU), garantizando que cada agregado pueda evolucionar de manera independiente. Los **Domain Services** aparecen como componentes coordinadores cuando una regla de negocio abarca más de un agregado sin pertenecer naturalmente a ninguno de ellos (por ejemplo, la asignación del almacén con menor carga de trabajo). La comunicación asíncrona y la notificación de cambios de estado entre módulos se representan mediante **Domain Events** (líneas punteadas con punta de flecha).



Fichas de Aggregates

Pedido

Raíz	**Pedido**
Contiene	ItemPedido, SubEnvio, PagoInfo, CancelacionInfo, DireccionEntrega
Responsabilidades	Controlar el ciclo de vida de la orden, validar montos, manejar división en sub-envíos, registrar cancelaciones y cierres.
Invariantes	No se confirma pago si la revalidación de stock falla (RN-05). • Límite de 50 unidades por SKU (RN-07).

	• Pedidos >$15,000 MXN pasan a revisión antifraude obligatoria (RN-09). • Cancelación libre solo dentro de 2 horas (RN-11). • Cancelación tras "En tránsito" exige autorización de supervisor (RN-13). • Un pedido dividido se marca "Entregado" únicamente cuando todos sus sub-envíos han llegado (RN-16).
Estados	Creado, Pago Confirmado, En Revisión Antifraude, En Preparación, Empacado, En Tránsito, Entregado, Cancelado
Eventos	PedidoCreado, PagoConfirmado, PedidoDividido, PedidoEnTransito, PedidoEntregado, PedidoCancelado
Casos de uso	CU-01, CU-02, CU-03, CU-04, CU-05

Inventario

Raíz	**AlmacenStock**
Contiene	AlmacenId, SKU, ExistenciaFisica, ExistenciaReservada
Responsabilidades	Reservar stock temporalmente, liberar reservas, revalidar disponibilidad y ajustar existencias físicas por incidencia.
Invariantes	ExistenciaDisponible = ExistenciaFisica - ExistenciaReservada. • Solo el sistema modifica la existencia reservada; ajustes a física requieren Supervisor de Almacén con motivo (RN-15).
Eventos	StockReservado, ReservaLiberada, DiscrepanciaStockReportada
Casos de uso	CU-01, CU-02, CU-03

Devolución

Raíz	**Devolucion**
Contiene	PedidoId, ListaItems, Motivo, EvidenciaFotografica, AprobadoPorSupervisor
Responsabilidades	Validar ventana de devolución y solicitar aprobación cuando aplique.
Invariantes	Devolución estándar aplica dentro de 5 días hábiles con producto sin usar (RN-20). • Devolución fuera de ventana, sin empaque o por daño exige evidencia fotográfica y aprobación exclusiva de Supervisor (RN-21).
Eventos	DevolucionSolicitada, DevolucionAprobada, DevolucionRechazada
Casos de uso	CU-05

4. Service Layer (Capa de Aplicación)

El Service Layer es la puerta de entrada de los casos de uso. Funciona como el director de una orquesta: decide qué pasos se realizan y en qué orden, pero no contiene reglas de negocio. En esta etapa definimos los Application Services, qué recibe y devuelve cada uno, con quién colabora y qué reglas delega al dominio.

Tabla de Application Services

Servicio	Método (CU)	Entrada	Colaboradores	Salida	Reglas que delega
CheckoutApplicationService	iniciarCheckout(CU-01)	ClienteId, Ítems (SKU, cantidad), Dirección	CheckoutRepository, InventoryRepository, UnitOfWork	CheckoutSessionDto (Folio, expiración)	Inventario: RN-04 (Reserva 15 min), RN-07 (Límite 50 SKU).
PaymentApplicationService	procesarPago (CU-02)	CheckoutId, TokenPago, Monto	OrderRepository, InventoryRepository, PaymentGatewayAdapter, EvaluadorAntifraude, UnitOfWork	ResultadoPagoDto (Confirmado / Antifraude / Agotado)	Pedido/Inventario: RN-05 (Revalidación stock), RN-06 (Opciones agotados), RN-08 (3 intentos), RN-09 (Antifraude >$15k).
WarehouseApplicationService	asignarAlmacenYDividir(CU-03)	PedidoId	OrderRepository, WarehouseRepository, AsignadorDeAlmacen, UnitOfWork	AsignacionAlmacenDto (Almacén o Sub-envíos)	AsignadorDeAlmacen/Pedido: RN-14 (Cercanía/carga), RN-16 (División en sub-envíos).

Servicio	Método (CU)	Entrada	Colaboradores	Salida	Reglas que delega
CustomerSupportApplicationService	cancelarPedido(CU-04)	PedidoId, Solicitante, Motivo, Autorizador	OrderRepository, InventoryRepository, UnitOfWork	ConfirmacionCancelacionDto	Pedido: RN-11 (Autocancelación 2h), RN-12 (Asistida agente), RN-13 (Excepción supervisor post-tránsito).
CustomerSupportApplicationService	procesarDevolucion(CU-05)	PedidoId, Items, Motivo, Evidencia, Solicitante	OrderRepository, ReturnRepository, UnitOfWork	ResultadoDevolucionDto	Devolucion: RN-20 (Estándar 5 días), RN-21 (Excepcional con evidencia y supervisor).

Tabla de Solicitudes que Enruta cada Método

Método	Solicitud	Quién la Pide	Datos	Efecto
procesarPago	Pago Normal	Cliente	CheckoutId, Token	Confirma pago y pasa a preparación (RN-05).
	Pago Alto Monto	Sistema	CheckoutId, Monto >$15k	Pasa a "En Revisión Antifraude" por 24h (RN-09).

Método	Solicitud	Quién la Pide	Datos	Efecto
	Producto Agotado	Sistema	SKU agotado	Detiene cobro y presenta 3 opciones (RN-06).
cancelarPedido	Autocancelación	Cliente	PedidoId (<2h)	Cancela sin costo y libera stock (RN-11).
	Cancelación Asistida	Agente	PedidoId (Pre-tránsito)	Cancela y registra si es reintegro o merma (RN-12).
	Excepción Post-tránsito	Supervisor	PedidoId, Motivo	Cancela por excepción justificada de PetMarket (RN-13).
procesarDevolucion	Devolución Estándar	Cliente / Agente	PedidoId (<5 días, sin usar)	Aprobada por agente y genera instrucción de reembolso (RN-20).
	Devolución Excepcional	Supervisor	PedidoId, Evidencia fotográfica	Aprobada exclusivamente por supervisor (RN-21).

5. Repository y Unit of Work

El patrón Repository separa la lógica de negocio de la forma en que se guardan y recuperan los datos. La Unit of Work decide qué cambios se guardan juntos: o se confirma todo, o no se guarda nada.

Tabla de Repositorios

Repository	**Necesidad del Dominio**	**Métodos Propuestos**	**Objeto que Recupera o Guarda**
OrderRepository	Recuperar y guardar pedidos completos con sus sub-envíos y estados.	obtenerPorId(pedidoId), guardar(pedido)	Aggregate Pedido
InventoryRepository	Consultar y actualizar stock físico, reservado y disponible por SKU/almacén.	obtenerStock(sku, almacenId), guardarStock(almacenStock)	Aggregate AlmacenStock
ReturnRepository	Guardar y consultar solicitudes de devolución.	obtenerPorId(devolucionId), guardar(devolucion)	Aggregate Devolucion
CheckoutRepository	Administrar sesiones de checkout activas y reservas temporales.	obtenerPorId(checkoutId), guardar(session)	Aggregate CheckoutSession

Tabla de Unit of Work

Método de Servicio	**Cambios que Deben Completarse Juntos**	**Si Algo Falla**
iniciarCheckout	Sesión de checkout + Reserva en Inventario	Se deshace la reserva y no se crea la sesión de checkout.

Método de Servicio	**Cambios que Deben Completarse Juntos**	**Si Algo Falla**
procesarPago	Estado del Pedido (Confirmado) + Confirmación de Reserva de Inventario	Se revierte el pago tokenizado y el pedido permanece en checkout o se libera.
asignarAlmacenYDividir	Asignación de Almacén de Origen + Creación de Sub-envíos con Guías	No se guardan sub-envíos parciales; el pedido regresa a estado pendiente de asignación.
cancelarPedido	Estado del Pedido (Cancelado) + Liberación de Inventario + Registro de Merma	Ni cancelación ni liberación de inventario: se mantiene el estado previo (RN-11/12/13).
procesarDevolucion	Registro de Devolución + Actualización de Estado de Pedido + Orden de Reembolso	No se registra la devolución ni se emite reembolso parcial.

6. Commands y Queries: CQRS Básico

CQRS propone separar las operaciones que cambian el sistema (commands) de las que solo consultan información (queries).

Tabla de Commands

Tipo	**Nombre**	**Datos de Entrada**	**Handler y Dependencias**	**Resultado**	**Justificación**
Command	IniciarCheckoutCommand	ClienteId, Ítems, Dirección	IniciarCheckoutHandler -> InventoryRepository,	Folio checkout y	Inicia reserva de 15 min (RN-04).

Tipo	Nombre	Datos de Entrada	Handler y Dependencias	Resultado	Justificación
			CheckoutRepository, UnitOfWork	tiempo expiración	
Command	ProcesarPagoCommand	CheckoutId, TokenPago	ProcesarPagoHandler -> OrderRepository, InventoryRepository, PaymentGatewayAdapter, UnitOfWork	Estado resultado pago	Revalida stock y procesa cobro (RN-05, RN-09).
Command	DividirPedidoCommand	PedidoId	DividirPedidoHandler -> OrderRepository, WarehouseRepository, UnitOfWork	Folios y guías de sub-envíos	Genera sub-envíos cuando ningún almacén cubre el 100% (RN-16).
Command	CancelarPedidoCommand	PedidoId, Solicitante, Motivo, Autorizador	CancelarPedidoHandler -> OrderRepository, InventoryRepository, UnitOfWork	Confirmación cancelación	Aplica reglas de cancelación según estatus y rol (RN-11, 12, 13).
Command	SolicitarDevolucionCommand	PedidoId, Ítems,	SolicitarDevolucionHandler ->	Estado de aprobación devolución	Aplica reglas de devolución

Tipo	Nombre	Datos de Entrada	Handler y Dependencias	Resultado	Justificación
		Motivo, Evidencia	ReturnRepository, OrderRepository, UnitOfWork		estándar o por excepción (RN-20, 21).

Tabla de Queries

Tipo	Nombre	Datos de Entrada	Handler y Dependencias	Resultado	Justificación
Query	ConsultarEstatusPedidoQuery	PedidoId	ConsultarEstatusPedidoHandler -> ConsultasPedidosDao	EstatusPedidoDto (con sub-envíos y guías)	Permite al cliente ver el estatus real en app/web sin llamar a soporte (US-19).
Query	ObtenerReporteDiscrepanciaQuery	Fecha, AlmacenId	ObtenerReporteDiscrepanciaHandler -> ConsultasInventarioDao	ReporteDiscrepanciaDto	Reporte diario para el Gerente de Operaciones (Sección 10 BRD).
Query	ConsultarKPIsOperativosQuery	Mes, Año	ConsultarKPIsOperativosHandler -> ConsultasKPIsDao	TableroKPIsDto	Tablero mensual de tiempos de ciclo, cancelaciones y

Tipo	Nombre	Datos de Entrada	Handler y Dependencias	Resultado	Justificación
					devoluciones (Sección 10 BRD).

Tabla de DTOs (Modelo de Lectura)

DTO	Campos	Por qué NO es el Modelo de Escritura
EstatusPedidoDto	Folio, estatus general, lista de sub-envíos (folio, almacén, guía, fecha estimada, estatus)	Es plano y optimizado para UI; no expone reglas de dominio ni métodos de modificación de estado.
ReporteDiscrepanciaDto	SKU, Almacén, existencia física, existencia web, porcentaje discrepancia	Muestra datos agregados para el Gerente de Operaciones; no permite alterar existencias directas.
TableroKPIsDto	Tiempo de ciclo promedio, % pedidos divididos, tasa cancelación, tasa devolución	Objeto de lectura consolidada para toma de decisiones gerenciales.

7. Revisión con Principios SOLID

Tabla de Revisión SOLID

Principio	Pregunta Propuesta	Respuesta con Evidencia de Nuestro Diseño
S	¿Cada clase o servicio tiene una razón principal para cambiar?	Sí: 4 Application Services separados (Checkout, Payment, Warehouse, CustomerSupport) y un Handler por Command.
O	¿Se puede agregar una variante de regla sin editar condicionales?	Sí: las estrategias de opciones ante producto agotado (eliminar, esperar, sustituir) se implementan de forma extensible.
L	¿Las implementaciones se pueden sustituir sin cambiar comportamiento?	Sí: las implementaciones de repositorios (en memoria o SQL) cumplen los mismos contratos de excepción del dominio.
I	¿Cada consumidor depende solo de lo que usa?	Sí: los handlers de consulta dependen de interfaces de lectura (Consultas...Dao) y no de repositorios de escritura.
D	¿Aplicación y dominio dependen de abstracciones?	Sí: solo conocen interfaces (IOrderRepository, IPaymentGatewayAdapter, IUnitOfWork). No hay SQL en el dominio.

Hallazgos de la Revisión SOLID

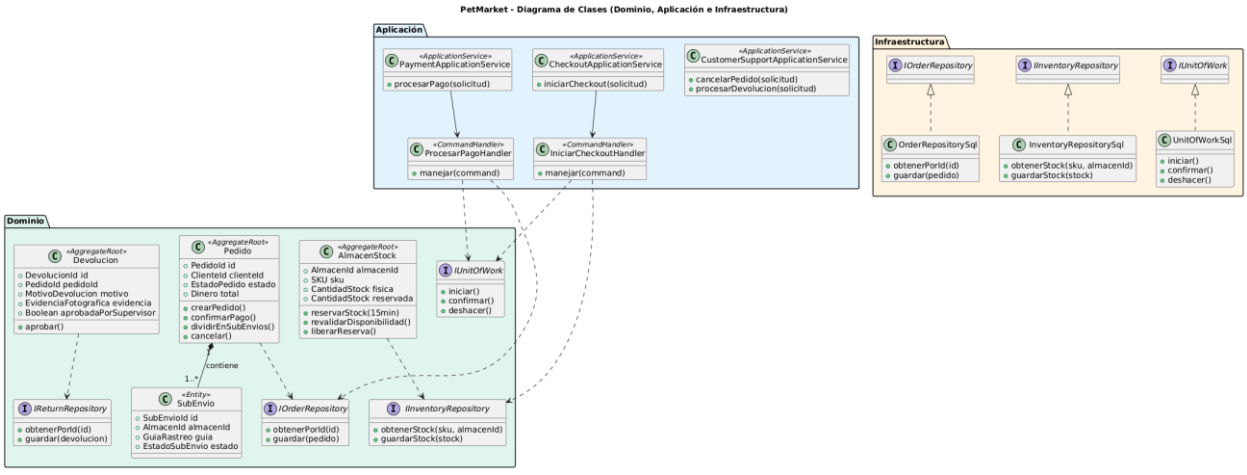
- **H1 (S - Confirmación):** Los servicios de atención al cliente están separados de la pasarela de pagos e inventario.
- **H2 (O - Corrección):** Las opciones ante productos agotados usaban condicionales rígidos. Se refactorizó usando un patrón de estrategia para permitir nuevas opciones a futuro.

- **H3 (L - Corrección):** Se definió un contrato explícito en IOrderRepository para que obtenerPorId lance siempre una excepción de negocio PedidoNoEncontradoException independiente del ORM.
- **H4 (I - Confirmación):** Las consultas de seguimiento del cliente usan interfaces de lectura independientes que no exponen métodos de guardado.
- **H5 (D - Confirmación):** La pasarela de pagos se desacopló mediante la interfaz IPaymentGatewayAdapter.

8. Diagramas UML y ADRs

Descripción del Diagrama de Clases UML

El Diagrama de Clases UML estructura formalmente la arquitectura Hexagonal / Cebolla adoptada para PetMarket. Se divide visualmente en tres paquetes principales representados por colores: **Dominio** (Verde), **Aplicación** (Azul) e **Infraestructura** (Naranja). Las dependencias fluyen estrictamente desde el exterior hacia el centro: la capa de Infraestructura implementa los puertos e interfaces definidos en la capa de Aplicación y Dominio. Se observan los agregados principales (Pedido, AlmacenStock, Devolucion), sus Value Objects inmutables (SKU, Dinero, Direccion), las interfaces de repositorio y la UnitOfWork que coordina la persistencia transaccional.



Codigo
@startuml
title PetMarket - Diagrama de Clases (Dominio, Aplicación e Infraestructura)
skinparam shadowing false
skinparam defaultFontName Arial

package "Dominio" #E1F5EE {

```
class Pedido <<AggregateRoot>> {
  +PedidoId id
  +ClienteId clienteId
  +EstadoPedido estado
  +Dinero total
  +crearPedido()
  +confirmarPago()
  +dividirEnSubEnvios()
  +cancelar()
}

class SubEnvio <<Entity>> {
  +SubEnvioId id
  +AlmacenId almacenId
  +GuiaRastreo guia
  +EstadoSubEnvio estado
}

class AlmacenStock <<AggregateRoot>> {
  +AlmacenId almacenId
  +SKU sku
  +CantidadStock fisica
  +CantidadStock reservada
  +reservarStock(15min)
  +revalidarDisponibilidad()
  +liberarReserva()
}

class Devolucion <<AggregateRoot>> {
  +DevolucionId id
  +PedidoId pedidoId
  +MotivoDevolucion motivo
  +EvidenciaFotografica evidencia
  +Boolean aprobadaPorSupervisor
  +aprobar()
}

interface IOrderRepository {
  +obtenerPorId(id)
  +guardar(pedido)
}

interface IInventoryRepository {
  +obtenerStock(sku, almacenId)
```

```
    +guardarStock(stock)
  }

  interface IReturnRepository {
    +obtenerPorId(id)
    +guardar(devolucion)
  }

  interface IUnitOfWork {
    +iniciar()
    +confirmar()
    +deshacer()
  }

  Pedido "1" *-- "1..*" SubEnvio : contiene
  Pedido ..> IOrderRepository
  AlmacenStock ..> IInventoryRepository
  Devolucion ..> IReturnRepository
}

package "Aplicación" #E3F2FD {
  class CheckoutApplicationService <<ApplicationService>> {
    +iniciarCheckout(solicitud)
  }
  class PaymentApplicationService <<ApplicationService>> {
    +procesarPago(solicitud)
  }
  class CustomerSupportApplicationService <<ApplicationService>> {
    +cancelarPedido(solicitud)
    +procesarDevolucion(solicitud)
  }

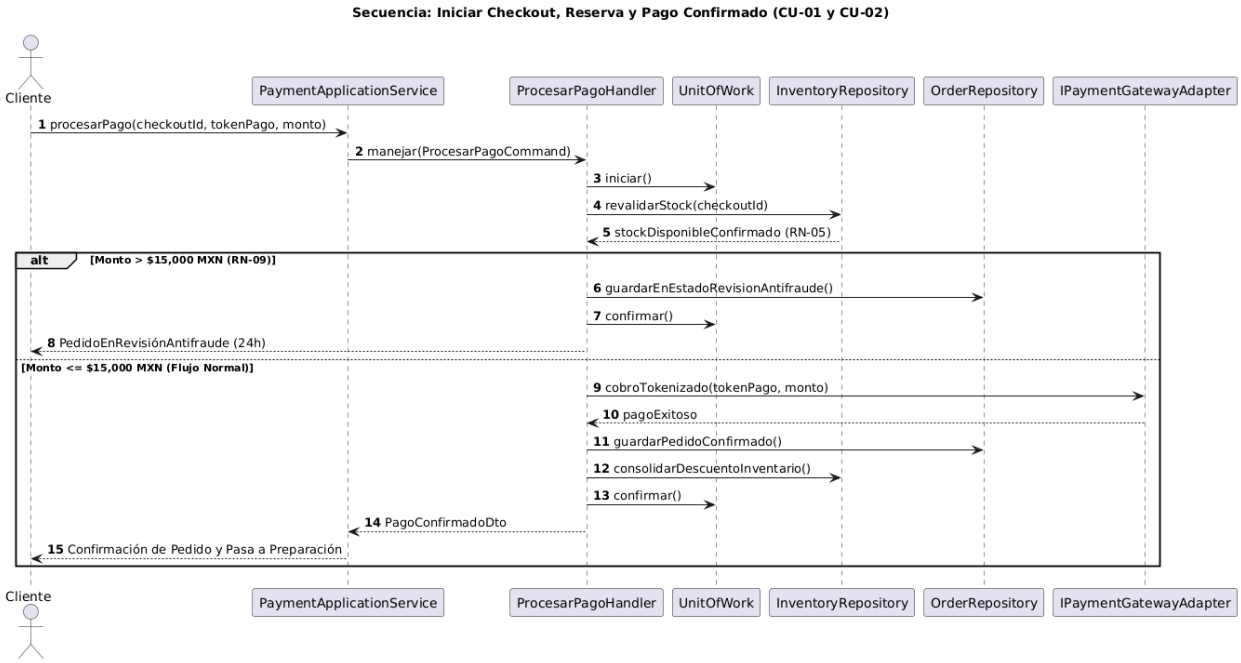
  class IniciarCheckoutHandler <<CommandHandler>> {
    +manejar(command)
  }
  class ProcesarPagoHandler <<CommandHandler>> {
    +manejar(command)
  }

  CheckoutApplicationService --> IniciarCheckoutHandler
  PaymentApplicationService --> ProcesarPagoHandler
  IniciarCheckoutHandler ..> IInventoryRepository
  IniciarCheckoutHandler ..> IUnitOfWork
  ProcesarPagoHandler ..> IOrderRepository
```

```
  ProcesarPagoHandler ..> IUnitOfWork
}

package "Infraestructura" #FFF3E0 {
  class OrderRepositorySql implements IOrderRepository {
    +obtenerPorId(id)
    +guardar(pedido)
  }
  class InventoryRepositorySql implements IInventoryRepository {
    +obtenerStock(sku, almacenId)
    +guardarStock(stock)
  }
  class UnitOfWorkSql implements IUnitOfWork {
    +iniciar()
    +confirmar()
    +deshacer()
  }
}
@enduml
```

1. **Flujo Normal (Checkout y Confirmación de Pago):** Ilustra el ciclo de vida feliz de una compra. El cliente inicia checkout a través de CheckoutApplicationService, el cual valida el límite por SKU y reserva existencias por 15 minutos en InventoryRepository mediante la UnitOfWork. Posteriormente, al procesar el pago, PaymentApplicationService revalida el stock en tiempo real, ejecuta el cobro tokenizado mediante IPaymentGatewayAdapter, confirma el pedido y emite el evento PagoConfirmado.



Código

```
@startuml
title Secuencia: Iniciar Checkout, Reserva y Pago Confirmado (CU-01 y CU-02)
skinparam shadowing false
skinparam defaultFontName Arial
autonumber

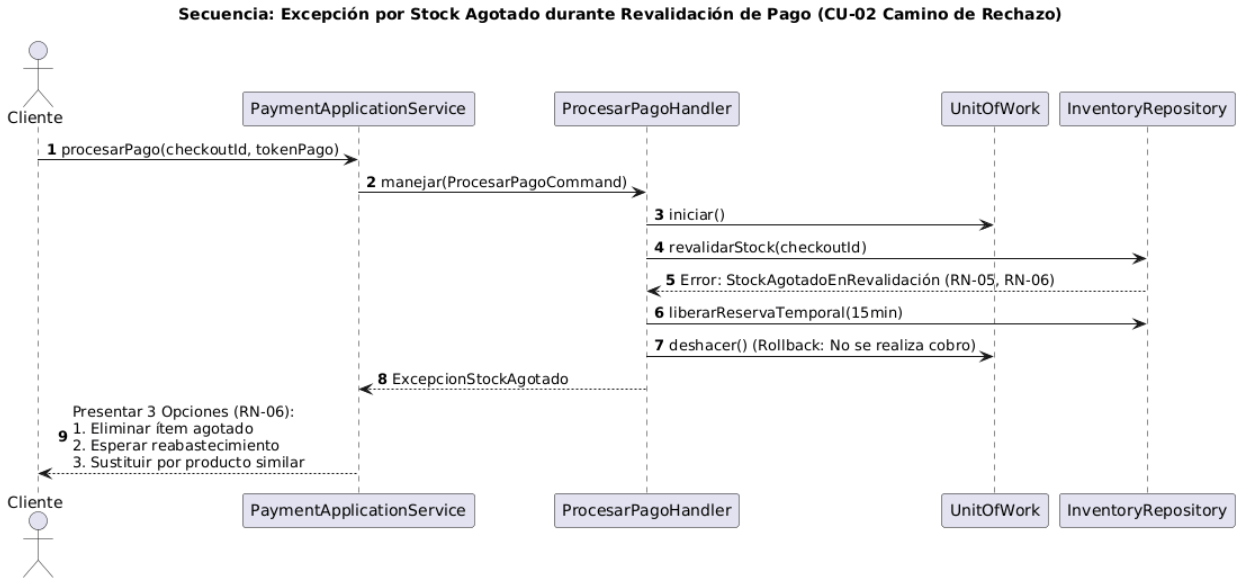
actor Cliente
participant "PaymentApplicationService" as PAS
participant "ProcesarPagoHandler" as PPH
participant "UnitOfWork" as UOW
participant "InventoryRepository" as IR
participant "OrderRepository" as OR
participant "IPaymentGatewayAdapter" as PGA

Cliente -> PAS : procesarPago(checkoutId, tokenPago, monto)
PAS -> PPH : manejar(ProcesarPagoCommand)
PPH -> UOW : iniciar()
PPH -> IR : revalidarStock(checkoutId)
IR --> PPH : stockDisponibleConfirmado (RN-05)

alt Monto > $15,000 MXN (RN-09)
    PPH -> OR : guardarEnEstadoRevisionAntifraude()
    PPH -> UOW : confirmar()
    PPH --> Cliente : PedidoEnRevisiónAntifraude (24h)
else Monto <= $15,000 MXN (Flujo Normal)
```

```
    PPH -> PGA : cobroTokenizado(tokenPago, monto)
    PGA --> PPH : pagoExitoso
    PPH -> OR : guardarPedidoConfirmado()
    PPH -> IR : consolidarDescuentoInventario()
    PPH -> UOW : confirmar()
    PPH --> PAS : PagoConfirmadoDto
    PAS --> Cliente : Confirmación de Pedido y Pasa a Preparación
end
@enduml
```

2. **Flujo con Excepción (Stock Agotado al Revalidar):** Ilustra el manejo de excepciones cuando un producto se agota durante el lapso de checkout. PaymentApplicationService detecta la discrepancia al ejecutar revalidarStock(), detiene la transacción sin realizar el cobro en la pasarela, libera la reserva previa y retorna el objeto ResultadoPagoDto con las tres opciones de resolución para el cliente (eliminar ítem, esperar reabastecimiento o sustituir).



Código

```
@startuml
title Secuencia: Excepción por Stock Agotado durante Revalidación de Pago (CU-02 Camino de Rechazo)
skinparam shadowing false
skinparam defaultFontName Arial
autonumber

actor Cliente
participant "PaymentApplicationService" as PAS
```

```
participant "ProcesarPagoHandler" as PPH
participant "UnitOfWork" as UOW
participant "InventoryRepository" as IR

Cliente -> PAS : procesarPago(checkoutId, tokenPago)
PAS -> PPH : manejar(ProcesarPagoCommand)
PPH -> UOW : iniciar()
PPH -> IR : revalidarStock(checkoutId)
IR --> PPH : Error: StockAgotadoEnRevalidación (RN-05, RN-06)

PPH -> IR : liberarReservaTemporal(15min)
PPH -> UOW : deshacer() (Rollback: No se realiza cobro)
PPH --> PAS : ExcepcionStockAgotado

PAS --> Cliente : Presentar 3 Opciones (RN-06):\n1. Eliminar ítem agotado\n2. Esperar
reabastecimiento\n3. Sustituir por producto similar
@enduml
```

Tablas de Architecture Decision Records (ADRs)

ADR-01: Patrón de Lógica de Negocio

Campo	**Contenido**
Título	Domain Model como patrón principal de lógica de negocio en PetMarket
Estado	Aceptado
Contexto	PetMarket maneja 21 reglas de negocio complejas, operaciones multialmacén, reservas temporales de 15 min, revalidación inmediata pre-cobro, división automática en sub-envíos y procesos de cancelación/devolución jerárquicos según rol.
Alternativas	1. *Transaction Script:* Rápido de implementar pero generaría duplicación masiva de validaciones en scripts procedurales.

Campo	Contenido
	2. *Table Module:* Adecuado para consultas pero ataría la lógica a tablas sin modelar transiciones de estado. 3. *Domain Model:* Encapsula estado e invariantes en entidades ricas.
Decisión	Adoptar **Domain Model** como patrón central para el núcleo transaccional.
Consecuencias	**Beneficios:** Encapsulamiento total de reglas (RN-01 a RN-21) en agregados protegidos. **Costos/Riesgos:** Mayor esfuerzo inicial de diseño y mapeo ORM.
Trazabilidad	RN-01 a RN-21, US-01 a US-21, CU-01 a CU-05.

ADR-02: Repositorios Desacoplados y Control Transaccional con Unit of Work

Campo	Contenido
Título	Repositorios orientados a Agregados y Unit of Work por solicitud
Estado	Aceptado
Contexto	Se debe garantizar la consistencia atómica entre la reserva de inventario, el cobro en pasarela externa y la creación de sub-envíos, sin acoplar el dominio a la base de datos SQL.

Campo	Contenido
Alternativas	1. *Acceso directo SQL en Servicios:* Altamente acoplado. 2. *Repositorios independientes sin Unit of Work:* Riesgo de persistencias parciales en fallos de red. 3. *Repository + Unit of Work:* Garantiza atomicidad y aislamiento de persistencia.
Decisión	Definir interfaces de Repositorio en el Dominio y manejar la atomicidad con una UnitOfWork por solicitud de comando.
Consecuencias	**Beneficios:** Transacciones "todo o nada" que evitan sobreventa de stock. **Costos/Riesgos:** Creación de múltiples interfaces y abstracciones.
Trazabilidad	RN-04, RN-05, RN-16, Hallazgos H-03 y H-04.

ADR-03: CQRS Básico con DTOs de Lectura

Campo	Contenido
Título	CQRS básico con separación de Handlers, DTOs planos y una sola base de datos
Estado	Aceptado
Contexto	La aplicación web y móvil requiere consultar frecuentemente el estatus detallado de

Campo	Contenido
	pedidos y sub-envíos (US-19) y generar reportes operativos sin bloquear las entidades transaccionales de escritura.
Alternativas	1. *Modelo único:* Consultar directamente cargando Agregados pesados. 2. *CQRS Completo con bases duplicadas:* Excesiva complejidad técnica. 3. *CQRS Básico:* Separación de código y DTOs planos sobre la misma base de datos.
Decisión	Implementar **CQRS Básico** mediante Commands (Handlers transaccionales) y Queries (Consultas optimizadas con DTOs).
Consecuencias	**Beneficios:** Consultas de alto rendimiento para el cliente y reportes sin sobrecargar el dominio. **Costos/Riesgos:** Mantenimiento de clases DTO adicionales.
Trazabilidad	US-19, US-20, US-21, Sección 10 del BRD.

9. Validación de la Propuesta

Tabla de Validación por Caso de Uso

CU	¿Quién Recibe la Solicitud?	¿Dónde se Valida cada Regla?	¿Qué Objetos Cambian?	¿Cómo se Persiste?	¿Qué se Devuelve?
CU-01	CheckoutApplicationService -> IniciarCheckoutHandler	Aggregate CheckoutSession e Inventario (RN-04, RN-07)	CheckoutSession nueva y AlmacenStock (reserva aumenta)	CheckoutRepository y InventoryRepository en una UnitOfWork	Folio de checkout y expiración de 15 min.
CU-02	PaymentApplicationService -> ProcesarPagoHandler	Aggregate Pedido, Inventario y EvaluacionRiesgo (RN-05, RN-08, RN-09)	Pedido (pasa a Pago Confirmado o Revisión Antifraude)	OrderRepository y InventoryRepository en UnitOfWork	Estado del pago o presentación de opciones si agotado.
CU-03	WarehouseApplicationService -> DividirPedidoHandler	Domain Service AsignadorDeAlmacen y Aggregate Pedido (RN-14, RN-16)	Pedido (crea colección de SubEnvio)	OrderRepository en UnitOfWork	Lista de sub-envíos con folios y guías de rastreo.
CU-04	CustomerSupportApplicationService -> CancelarPedidoHandler	Aggregate Pedido (RN-11, RN-12, RN-13 según estatus y rol)	Pedido (estado Cancelado) e Inventario (libera reserva/stock)	OrderRepository e InventoryRepository en UnitOfWork	Confirmación de cancelación. Error si requiere supervisor.
CU-05	CustomerSupportApplicationService ->	Aggregate Devolucion (RN-20, RN-21 según	Devolucion nueva y Pedido	ReturnRepository y OrderReposit	Estado de aprobación de la devolución.

CU	¿Quién Recibe la Solicitud?	¿Dónde se Valida cada Regla?	¿Qué Objetos Cambian?	¿Cómo se Persiste?	¿Qué se Devuelve?
	SolicitarDevolucionHandler	plazo, empaque y rol)		ory en UnitOfWork	

Tabla de Validación de la Guía

Validación	Pregunta	Resultado del Equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	Sí. Todos los componentes se trazan directamente con las 21 RNs y la Sección 10 del BRD.
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	Sí. Se mantiene el lenguaje ubicuo (*SKU*, *Sub-envío*, *Reserva*, *Pedido Padre*).
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido?	Sí. La Service Layer solo coordina; la lógica de revalidación, plazas y permisos vive en el Dominio.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	Sí. Todos los participantes de las secuencias coinciden con las clases del diagrama de UML.
Persistencia	¿El dominio evita depender directamente de SQL u ORM?	Sí. El dominio interactúa exclusivamente con interfaces abstractas de repositorios.

Validación	Pregunta	Resultado del Equipo
Transacción	¿El límite de la operación y su manejo de error están definidos?	Sí. Una UnitOfWork por solicitud con rollback completo ante excepciones.
IA	¿Cada aporte de IA fue revisado y su uso está registrado?	Sí. Registrado detalladamente en la bitácora de IA.

Bitácora de IA

Tabla de Iteraciones de la Bitácora

Iteración	Objetivo	Prompt (Resumen)	Respuesta Útil	Error o Supuesto Detectado	Decisión del Equipo
1	Seleccionar patrón de lógica de negocio para PetMarket.	"Compara Transaction Script, Table Module y Domain Model para PetMarket considerando sus 21 reglas de negocio."	Sugirió que Domain Model es ideal para proteger la consistencia de inventario y reservas.	La IA inicialmente subestimó el esfuerzo de ORM para mapear sub-envíos.	Se aceptó Domain Model agregando control de transacciones estricto con Unit of Work.
2	Diseñar la Service Layer para checkout y pagos.	"Propón métodos de Application Services para checkout, revalidación y pagos de PetMarket sin	Generó firmas limpias con DTOs de entrada y salida.	La IA intentó poner la validación de stock dentro del servicio de aplicación.	Se corrigió para delegar toda validación de stock al agregado Inventario.

Iteración	Objetivo	Prompt (Resumen)	Respuesta Útil	Error o Supuesto Detectado	Decisión del Equipo
		lógica de negocio."			
3	Estructurar los Aggregates y Entidades del Mapa de Dominio.	"A partir del glosario de PetMarket, define los Aggregates principales y sus invariantes."	Identificó los agregados Pedido, Inventario y Devolucion.	La IA omitió el concepto de "Sub-envío" como entidad del pedido.	Se ajustó el agregado Pedido para incluir la colección de SubEnvio cumpliendo RN-16.
4	Evaluar la arquitectura propuesta contra SOLID.	"Analiza las clases y servicios propuestos para PetMarket y detecta violaciones SOLID."	Señaló que las opciones ante producto agotado usaban condicionales rígidos.	Sugirió crear un aggregate independiente para cada sub-envío.	Se rechazó crear agregados por sub-envío para mantener la consistencia en el pedido padre.

Tabla de Errores Cometidos por la IA

#	Error	Cómo se Detectó	Corrección
1	Colocó la regla de revalidación de stock en el PaymentApplicationService.	Revisión del principio S de SOLID y guía de la actividad.	Se movió la lógica de revalidación a la entidad Inventario / Pedido.

#	Error	Cómo se Detectó	Corrección
2	Omitió el concepto de Pedido Padre y Sub-envíos en el mapa de dominio.	Validación contra la RN-16 del BRD.	Se agregó SubEnvio como entidad contenida en el agregado Pedido.
3	Sugirió implementar transacciones separadas por sub-envío.	Verificación de consistencia del pedido.	Se mantuvo una única UnitOfWork por comando para garantizar atomicidad.
4	Generó diagramas en sintaxis Mermaid en lugar de PlantUML.	Revisión de requisitos de la entrega.	Se tradujo completamente el código a sintaxis PlantUML.

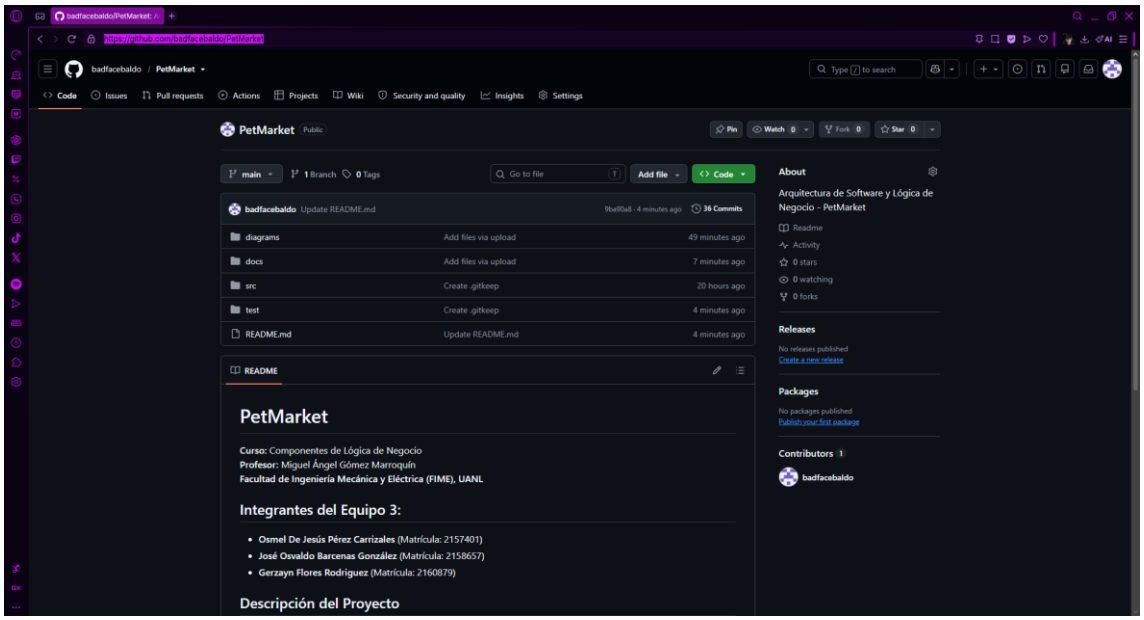
Tabla de Decisiones del Equipo

Decisión	Efecto
Seleccionar *Domain Model* como patrón principal	Encapsula las 21 reglas de negocio y protege las invariantes de inventario.
Mantener SubEnvio dentro del agregado Pedido	Garantiza que el pedido padre solo cambie a "Entregado" cuando todos sus sub-envíos lleguen (RN-16).
Implementar UnitOfWork por solicitud	Previene estados inconsistentes entre cobros de pasarela y reserva de stock.
Aplicar CQRS Básico con DTOs planos	Optimiza el rastreo del cliente en app/web sin sobrecargar los agregados de dominio.

Repositorio de GitHub

Todos los entregables se organizaron en el repositorio oficial del equipo con la siguiente estructura:

https://github.com/badfacebaldo/PetMarket



Conclusiones Individuales

José Osvaldo Barcenas González (Matrícula: 2158657)

El desarrollo de esta arquitectura para PetMarket me permitió comprender la importancia vital de diseñar y proteger las reglas de negocio en el núcleo del dominio antes de escribir código. Gracias al uso del modelo de dominio, logramos que reglas complejas como la reserva de 15 minutos en el checkout, la revalidación inmediata del stock antes de cobrar y la división automática de pedidos en sub-envíos queden encapsuladas y consistentes, evitando la dispersión de lógica en scripts o controladores. El uso responsable de la inteligencia artificial como copiloto aceleró la estructuración de la Service Layer y los repositorios, pero la revisión humana rigurosa fue indispensable para garantizar la alineación con los requerimientos del BRD de la Unidad 2.

Osmel De Jesús Pérez Carrizales (Matrícula: 2157401)

Implementar una arquitectura basada en capas con separación estricta entre dominio, aplicación e infraestructura nos otorgó una flexibilidad enorme para PetMarket. La aplicación del patrón Unit of Work junto con los repositorios desacoplados garantiza que operaciones críticas como la confirmación de pagos y la gestión de mermas o devoluciones mantengan la atomicidad transaccional ante fallos de red. Asimismo, la separación de comandos y consultas mediante un CQRS básico optimiza la visibilidad

en tiempo real del estatus de los pedidos para los clientes, cumpliendo con las metas de reducción de llamadas a soporte técnico establecidas en el proyecto.

Gerzayn Flores Rodriguez (Matrícula: 2160879)

Esta actividad práctica cerró la brecha entre el análisis conceptual de la Unidad 2 y la implementación técnica de la Unidad 3. Al aplicar los principios SOLID y estructurar adecuadamente los agregados, logramos resolver problemas complejos del negocio como la gestión de excepciones en devoluciones y las revisiones antifraude para montos elevados. La documentación mediante ADRs y la bitácora de IA reflejan el rigor metodológico del equipo al evaluar alternativas, aceptar costos de complejidad inicial justificados y mantener una trazabilidad impecable desde las necesidades del negocio hasta los componentes del software.